(*) Dense Retrieval

In the previous chapter, you used keyword search to query Wikipedia using two different queries:

- Who discovered penicillin?
- Who was the first person to win two Nobel prizes?

Keyword search did well with the first query, but not with the second one. The reason is that the second one has very popular words, which may appear in many documents that don't necessarily contain the answer to the question. A way to obtain better results is to have the model 'understand' what the question is actually asking. This is where "semantic search" comes into play. Semantic search is the ability to search by meaning - not just keyword matching. Language models enable two of the main ways of semantic search: Dense retrieval, & reranking. The concepts of dense retrievals were illustrated in a chapter earlier in this module. ~~So feel~~ Here, you'll see dense ~~ds~~ retrieval in action, with the same Wikipedia dataset & the same queries that you used for keyword search.

↳ Querying the Dataset Using Dense Retrieval.

Dense retrieval uses a text embedding in order to search for documents that are similar to a query. ~~Df~~ Embeddings assign a vector (long list of numbers) to each piece of text. One of the main properties in an embeddings is that similar pieces of text go to similar vectors.

ch to query

d prizes?

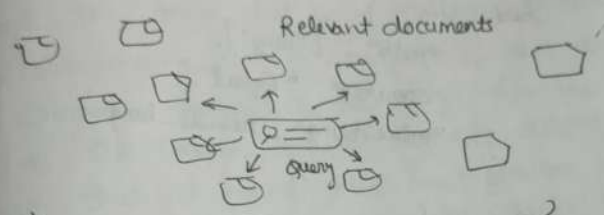
not with
as very
nts that
on. A way
derstand'
" semantic
lity to
language
search:
nse
in this
al in
same

al.

search
ngs
f text
imilar

short, dense retrieval consists of the following:

- Finding the embedding vector corresponding to the query
- Finding the embedding vectors corresponding to each of the responses (in this case, Wikipedia articles)
- Retrieving the response vectors that are closest to the query vector in the embedding.



Dense retrieval finds the closest documents to the query in the embedding

To use dense retrieval, we'll first define the following function which uses the Weaviate client we defined in the previous chapter. Just like with keyword search, we'll tell the vector database what properties we want from each retrieved document, and filter them to the English language (using results lang)

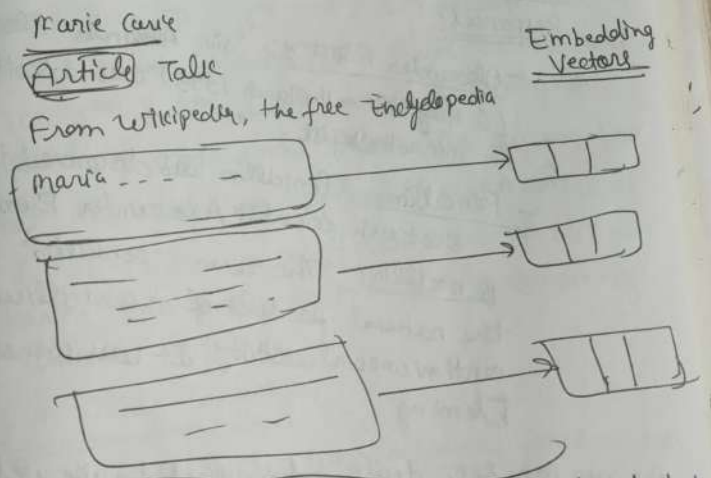
```
def dense_retrieval (query, results_lang = 'en', num_results=10):
    nearText = { "concepts": [query]}
    properties = [ "text", "title", "url", "views", "lang",
                   "_additional {distance} "]

    # To filter by language
    where_filter = {
            "path" : ["lang"],
            "operator": "Equal",
            "valueString" : results_lang

        }
    response = (
            client.query.
            .get ("Articles", properties)
            .with_near_text (nearText)
            .with_where (where_filter)
            .with_limit (num_results)
            .do()
    )
    result = response['data']['Get']['Articles']
    return result.
```

Chunking the Articles

This process of finding the closest documents to a query in an embedding will yield good results. However, articles may be very long & this could make things complicated. In order to have more granularity, we'll split the articles by paragraph. This means that we'll find the embedding vector corresponding to each paragraph of each article in the Wikipedia dataset. In that way, when the model retrieves the answer, it will actually output the paragraph that it found the most similar to the query, as well as the article in which this paragraph belongs.



Wikipedia articles get chunked by paragraph, and each chunk gets assigned an embedding vector.

↳ Back to Querying the Dataset

Let's review the two queries we used in the previous chapter

- Simple query : "Who discovered penicillin?"
- Hard query : "What was the first person to win two Nobel prizes?"

Now, let's look at the 3 top results for each query using dense retrieval. Recall that the responses here are at the paragraph level, so the model may sometimes retrieve the same article several times by outputting different paragraphs from the same article.

↳ Query ①: "Who discovered pencillin?"

Responses:

- Alexander Fleming: "Sir Alexander Fleming (6 Aug 1881 - 11 March 1995) was a Scottish physician & microbiologist ..."
- Pencillin: "Penicillin was discovered in 1928 by scottish scientist Alexander Fleming ..."
- Penicillin: "The term "penicillin" is defined as the natural product of "penicillicum" mould with antimicrobial activity. It was coined by Alexander Fleming ..."

As you can see, dense retrieval did quite well by finding paragraphs that contain the exact answer.

Query ②: "Who was the first person to win two Nobel prizes?"

Answers:

- Nobel prize in literature: "The Nobel prize in literature can be shared by two individuals..."
- Nobel prize: "Although posthumous nominations are not presently permitted, ..."
- Nobel prize: "Few people have received 2 Nobel prizes. Marie Curie received the Physics prize..."
- Marie Curie: "Marie Curie was the first woman to win a Nobel prize, the first person to win two Nobel prizes, ..."

As you can see, dense retrieval did much better than keyword search here. The 2nd, 3rd & 4th results are in the correct documents (Nobel prize & Marie Curie), and in fact, the 3rd & 4th results are in a paragraph which explicitly contains the answer. The reason for this is that the embedding captures the semantics of the text & is able to see if two pieces of text have a similar meaning, even if they don't necessarily share many words in common.

⇒ Searching in Other Languages

As you may have noticed, the dense_retrieval function has a parameter called results_lang. This parameter determines the language in which the search results are outputted.

It is defaulted to English ('en'), but for this demo, it can also be set to German ('de'), French ('fr'), Spanish ('es'), Italian ('it'), Japanese ('ja'), Arabic ('ar'), (Simplified) Chinese ('zh'), Korean ('ko') & Hindi ('hi'). However, the Cohere multilingual embedding handles over 100 languages.

For the first example, let's search for results to the English query "Who was the first person to win two Nobel prizes" in Arabic. The line of code is:

```
↳ arabic_results = dense_retrieval(hard_query, results_lang = 'ar')
```

As you can see in the lab, the top 3 are the following:

- Nobel Prize جائزة نوبل
- List of Nobel Laureates قائمة حائزي جائزة نوبل
- Women in Society المرأة

Now let's search for the French results to a query in Spanish. The query is "Quién descubrió la penicilina?" ("Who discovered penicillin?")

```
spanish_query = "Quien descubrio la penicilina?"
french_results = dense_retrieval(spanish_query, results_lang = 'fr')
```

The results are the following

- Pénicilline
- Pénicilline
- Alexander Fleming

Conclusion

Dense retrieval is a search method that uses a text embedding to find the semantically closest answers to a query. In our Wikipedia example, dense retrieval did quite well in 3 queries. In the next chapter, you will learn how to improve search results even further, with a reranking method.

Reranking

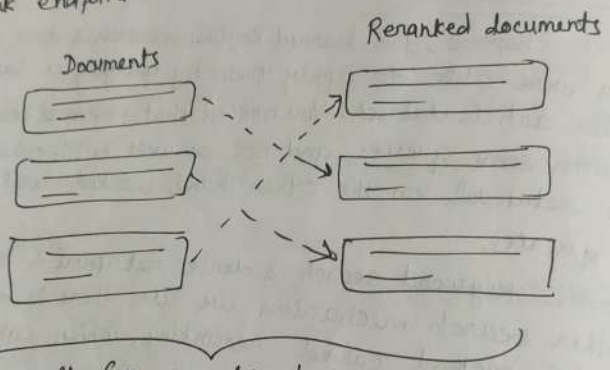
In prev. chapters, you learned keyword search & dense retrieval, and you were able to apply them by querying a large Wikipedia article dataset. You noticed that keyword search performed well with some queries, and not so well with others. Dense retrieval, on the other hand, worked well with all the queries.

For both, keyword search & dense retrieval & in fact, for any other search mechanism we use, there is a very powerful method called reranking, which can enhance it. With Cohere, you can perform reranking using Rerank endpoint.

Reranking works as follows: For each pair (query, response) it assigns a ~~rele~~ relevance score. As the name hints, relevance scores are high for pairs in which the response is relevant to the query, and low otherwise. In this chapter, you'll learn how to use Reranking to improve the Wikipedia search results you found previously in this module.

↳ Using Rerank to improve Keyword Search.

Reranking is a very powerful method which can significantly boost any existing search system. In short, reranking takes a query & a response, and outputs a relevance score b/w them. In that way, one can use any search system to surface a no. of documents that can potentially contain the answer to a query & then sort them using the Rerank endpoint.



The results from any search system get reranked based on their relevance to the query.

↳ Remember that the results we obtained for the query "Who was the first person to win two Nobel prizes" using the keyword-search function were the following:

Query: "Who was the first person to win two Nobel prizes?"

Responses:

- Neutrino
- Western culture
- Reality television.

These could contain the answer somewhere in the document, but they are certainly not the best documents for this query. Let's dig in a bit more, and find the first 100 results. To save space, I'll only note the top 20 titles:

1. Neutrino
2. Western culture
3. Reality television
4. Peter Mullan
5. Indiana Pacers
6. William Regal
7. Nobel Prize
8. " "
9. " "
10. Noble gas
11. Nobel prize in Literature
12. D.C. United
13. Nobel Prize in Literature
14. 2021-2022 Manchester United F.C. season
15. Nobel Prize
16. " "
17. Zach LaVine
18. 2011 Formula One World Championship
19. 2021-2022 Manchester United F.C. season
20. Christians.

OK, there's a high chance that the answer is there. Let's see if reranking can help us find it. The following function calls the Rerank endpoint. Its inputs are the query, the responses, & the number of responses we'd like to retrieve.

```
def rerank_responses(query, responses, num_responses=3):
    reranked_responses = co.rerank(
        query=query,
        documents=responses,
        top_n=num_responses,
        model='rerank-english-v3.0',
        return_documents=True
    )
    return reranked_responses
```

Rerank will output the result, as well as the relevance score. Let's look at the top 3 results.

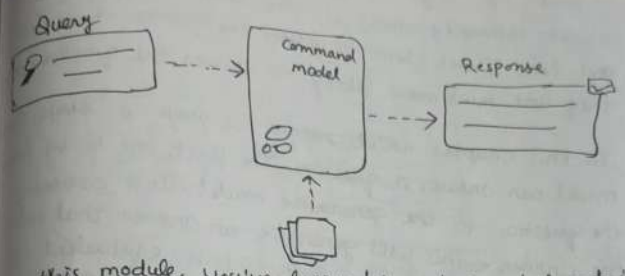
Query: "Who was the first person to win two Nobel prizes?"

Responses:

- Nobel Prize: "Five people have received two Nobel Prizes. Marie Curie received the ---"
  Relevance score: 1.00
- Nobel Prize: "In terms of the most prestegious awards in STEM fields, only a small ---"
  Relevance score: 0.97
- Nobel Prize in Literature: "There are also prizes for honouring the lifetime achievement of writers ---"
  Relevance score: 0.87

Well, that certainly improved the keyword search results! Even though the third result does not work, the first two retrieved the correct article that contains the answer.

Generating Answers



In this module, you've learned how to search & retrieve info^n from large databases in very effective ways. In this chapter, you'll learn how to combine this with a generative model, in order to get an answer in sentence format, instead of a list of search results.

Large Language Models are very good at answering questions, but they are prone to some limitations, such as incorrect info^n, or even hallucinations. A good way to fix this is to enhance an LLM with a search mechanism.

In short, this combination is done in the following way:

Given a query, the search mechanism retrieves one or more documents containing the answer.

These documents are given to the large language model, and it is instructed to generate an answer based on that info^n.

I like to imagine this the following way. If I have a question about thermodynamics, I can pick a random friend of mine, and ask them that question. They may or may not get the answer wrong.